



Universidad Nacional Autónoma de México
Facultad de Ciencias

Reporte 01

Esquemas de Codificación

No. de cuenta	Nombre
320148204	González Lucero Alan Uriel
321225087	Rivera Jiménez Isaac
321306102	Tapia Sánchez Jesús



Complejidad Computacional

Semestre 2027-1

Índice

1	Problema	2
2	Ejercicio 1	2
2.1	Solución	2
3	Ejercicio 2	3
3.1	Solución	3
4	Ejercicio 3	5
4.1	Solución	5
5	Ejercicio 4	6
5.1	Solución	6
6	Ejercicio 5 (Opcional)	7
6.1	Solución	7
7	Referencias Bibliográficas	7

Problema

Enunciado

“DeliCiencias” es un nuevo proyecto estudiantil de logística y entrega de comida a domicilio que busca competir optimizando las rutas de sus repartidores. **Cada repartidor** utiliza una mochila térmica de alta tecnología que tiene una **capacidad máxima de peso** estricta. En un día de alta demanda, el sistema central recibe decenas de pedidos simultáneos de distintos locales aliados (pizzas, ensaladas saludables, hamburguesas, etc.). **Cada pedido pendiente i** tiene un **peso físico w_i** (en gramos) y **representa una ganancia económica v_i** (en pesos) para el repartidor.

El objetivo es diseñar un esquema para representar el catálogo de pedidos pendientes en un archivo, y proponer un algoritmo que determine si un repartidor puede seleccionar un subconjunto de estos pedidos tal que quepan en su mochila térmica sin superar el límite de peso W , y que en conjunto le garanticen una ganancia de al menos V pesos.

Ejercicio 1

Enunciado

- (a) ¿Cómo podemos expresar este problema en su versión de búsqueda?
- (b) ¿Tiene sentido plantearse una versión de optimización?
- (c) Plantea el problema en su versión de decisión, y enúncialo en su forma canónica.

Solución

- (a) **Un problema de búsqueda** consiste en encontrar y devolver la estructura exacta que cumple con las condiciones establecidas.

Para el caso de “DeliCiencias”, la versión de búsqueda se expresaría formalmente de la siguiente manera:

Dado un catálogo de pedidos pendientes donde cada pedido i tiene un peso físico w_i y una ganancia económica v_i , una capacidad máxima de mochila W y una ganancia meta V , **encuentra y devuelve** un subconjunto específico de pedidos S tal que la suma de sus pesos no exceda W ($\sum_{i \in S} w_i \leq W$) y la suma de sus ganancias alcance o supere la meta V ($\sum_{i \in S} v_i \geq V$). Si no existe ninguna combinación que cumpla ambas condiciones, el algoritmo debe reportarlo.

- (b) Si, tiene sentido, ya que se busca maximizar la ganancia del repartidor aprovechando al máximo la capacidad W de la mochila, sin depender de un umbral fijo V .

La versión de optimización modificaría la estructura del problema al eliminar el parámetro de entrada V . El algoritmo ya no se limitaría a buscar una combinación que

simplemente cruce un umbral, sino que evaluaría el catálogo para encontrar el subconjunto exacto de pedidos cuya suma de pesos no exceda W y cuya suma de ganancias sea el máximo absoluto posible.

(c) **OPTIMIZACIÓN DE ENTREGAS - VERSIÓN DE DECISIÓN.**

Datos (parámetros): Un conjunto finito de pedidos pendientes $P = \{p_1, p_2, \dots, p_n\}$. Para cada pedido $p_i \in P$, un peso físico $w_i \in \mathbb{Z}^+$ y una ganancia económica $v_i \in \mathbb{Z}^+$. Una capacidad máxima de la mochila $W \in \mathbb{Z}^+$ y una ganancia meta $V \in \mathbb{Z}^+$.

Pregunta (objetivo): ¿Existe un subconjunto $S \subseteq P$ tal que la suma de los pesos de los pedidos en S sea menor o igual a W ($\sum_{p_i \in S} w_i \leq W$) y la suma de sus ganancias sea mayor o igual a V ($\sum_{p_i \in S} v_i \geq V$)?

Ejercicio 2

Enunciado

- (a) Describe un esquema de codificación para ejemplares del problema anterior (en su versión de decisión).

Nota: No es válido usar bibliotecas o herramientas externas que ya lean formatos predefinidos (como JSON, XML, CSV, etc).

- (b) Describe e implementa un algoritmo que lea archivos con el formato propuesto en el inciso anterior e imprima en consola los datos del ejemplar.

El programa debe recibir como entrada (desde consola):

- Nombre del archivo de entrada con el ejemplar a leer.

El ejemplar debe seguir el esquema de codificación propuesto

Como resultado de la ejecución, el programa deberá producir como salida:

- Imprimir en consola el total de pedidos disponibles
- Imprimir en consola la capacidad máxima de la mochila y la ganancia meta
- Imprimir dos ejemplos de pedidos disponibles, mostrando su peso y ganancia

Solución

- (a) Ya que el diseño de un esquema de codificación consiste en establecer reglas exactas para traducir los parámetros de un ejemplar a una cadena de texto que un programa pueda procesar, y debido a que en la práctica se nos prohíbe utilizar formatos estructurados ya establecidos, vamos a crear un formato posicional basado en saltos de línea y un delimitador simple (como el espacio).

Así que, para la versión de decisión, necesitamos almacenar el total de pedidos (n), la capacidad máxima (W), la ganancia meta (V), y las características de cada pedido (w_i y v_i).

Obteniendo la siguiente estructura para el archivo de texto plano:

- Línea 1: Un número entero n que define el total de pedidos disponibles.
- Línea 2: Un número entero W que defina la capacidad máxima de la mochila (en gramos).
- Línea 3: Un número entero V que define la ganancia meta (en pesos).
- Línea 4 a la $n + 3$: Dos números enteros separados por un espacio, correspondientes a w_i (pesos) y v_i (ganancias) del pedido i .

(b) **1. Descripción del Algoritmo**

Entrada: Leer desde los argumentos de la consola el nombre del archivo de texto.

Lectura: Abrir el archivo en modo lectura.

Decodificación (Parseo):

- Leer la línea 1 y asignarla a n (total de pedidos).
- Leer la línea 2 y asignarla a W (capacidad de la mochila).
- Leer la línea 3 y asignarla a V (ganancia meta).
- Crear una lista iterando desde la línea 4 en adelante. Por cada línea, separar los valores por el espacio en blanco para obtener el peso (w_i) y la ganancia (v_i).

Salida: Imprimir n , W , y V . Luego, acceder a los dos primeros elementos de la lista de pedidos pendientes e imprimir sus datos.

2. Implementación en Python

La implementación del algoritmo se encuentra en el archivo `lector.py`

Ejercicio 3

Enunciado

Supongamos que, debido a una promoción agresiva de la aplicación, todos los pedidos disponibles en el sistema tienen exactamente el mismo peso (por ejemplo, todos pesan 500 gramos), aunque la ganancia de cada uno sigue siendo diferente dependiendo de la distancia de entrega.

Describe e implementa un algoritmo eficiente para resolver el problema planteado, para este caso particular (pesos idénticos). La implementación del algoritmo debe hacer uso del esquema de codificación implementado en el inciso 2.

El programa implementado debe recibir como entrada (desde consola) el nombre del archivo que contiene el ejemplar a probar, y como salida deberá imprimir:

- Respuesta al problema de decisión planteado (SÍ o NO se puede alcanzar la ganancia meta sin romper la mochila).
- Salida adicional: Si la respuesta es SÍ, imprimir el peso total.

Incluir al menos 2 archivos de prueba (ejemplos de ejecución) para ambos programas (incisos 2 y 3), así como ejemplos visuales de los ejemplares con los que prueben su programa.

Al menos deben agregar ejemplos de los siguientes casos

- ejemplar en el que la respuesta sea un sí
- ejemplar en el que la respuesta sea un no

* En caso de que no se pueda construir alguno de los ejemplares, justifica detalladamente por qué y propón algún otro caso de prueba.

Solución

1. Análisis

Dado que cada pedido pesa exactamente lo mismo (llamémosle w_{const}), el límite de la mochila W ya no restringe combinaciones complejas de pesos, sino simplemente la **cantidad máxima de pedidos** que se pueden llevar.

La cantidad máxima de paquetes k que caben en la mochila se calcula con una división entera:

$$k = \left\lfloor \frac{W}{w_{const}} \right\rfloor$$

Al saber que solo podemos llevar a lo mucho k pedidos y que todos consumen exactamente el mismo espacio, la estrategia óptima es seleccionar los k pedidos que ofrezcan la mayor ganancia individual.

Diseño del Algoritmo Eficiente

Lectura de Datos: Se lee el archivo utilizando el esquema que diseñamos en el Ejercicio 2. Se asume que todos los pedidos tienen el mismo peso, por lo que tomamos ese valor como w_{const} .

Cálculo de Límite de Artículos: Se determina cuántos pedidos caben calculando $k = \lfloor W/w_{const} \rfloor$. Si resulta que k es mayor que el número total de pedidos disponibles n , ajustamos $k = n$ (el repartidor tiene espacio para llevar todo el catálogo).

Ordenamiento (Sorting): Se ordena la lista de ganancias de los pedidos en orden descendente (del más lucrativo al menos lucrativo).

Selección: Se toman los primeros k elementos de esta lista ordenada y se suman para obtener la ganancia máxima posible (V_{max}).

Evaluación Final:

- Si $V_{max} \geq V$, el algoritmo imprime “SÍ” y reporta el peso total transportado, el cual será exactamente la cantidad de artículos seleccionados multiplicada por w_{const} .
- Si $V_{max} < V$, el algoritmo imprime “NO”.

Ventaja de Complejidad En este caso donde los pesos son idénticos, la complejidad de nuestro algoritmo está dominada únicamente por el paso de ordenamiento de las ganancias, lo que nos da un tiempo de ejecución de $O(n \log n)$. Por lo que se puede considerar un algoritmo “eficiente”.

2. Implementación en Python

La implementación del algoritmo se encuentra en el archivo `mochila_pesos_identicos.py`

Ejercicio 4

Enunciado

Para que las terminales móviles de los repartidores consuman la menor cantidad de datos posible, el protocolo de red requiere un esquema de codificación estrictamente binario. El archivo con la información del ejemplar solo puede contener caracteres de texto '0' y '1' (sin espacios, comas ni saltos de línea). Describe e implementa un algoritmo, similar al del ejercicio 2, que decodifique este archivo binario e imprima los datos del ejemplar original.

Solución

Ejercicio 5 (Opcional)

Enunciado

Ejercicio opcional (obligatorio para los que trabajan en equipos de 4 integrantes)

- Describe e implementa un algoritmo que tome archivos del Ejercicio 2 y genere un archivo para el Ejercicio 4.
- Para resolver el problema general (cuando los pedidos tienen pesos diferentes), un alumno propone la siguiente idea: "Calcular el 'valor por gramo' (densidad) de cada pedido dividiendo su ganancia entre su peso. Ordenar los pedidos del más valioso por gramo al menos valioso. Meter los pedidos a la mochila en ese orden hasta que ya no quepa el siguiente. Si al terminar la ganancia es $\geq V$ responder SÍ".

¿Este enfoque siempre funciona de manera óptima? En caso de que no, diseña un ejemplar de prueba (un archivo) donde esta estrategia falle (es decir, el algoritmo diga que NO se alcanza la meta, pero en realidad SÍ existe una combinación válida), o justifica por qué este enfoque sí podría siempre la respuesta correcta.

Solución

Referencias Bibliográficas

Referencias